

Електронски факултет у Нишу

Примена вештачке интелигенције за детекцију емоција у људском говору

Студент: Данило Милошевић

Професор: Јелена Николић

Садржај

1. Увод **3**
1.1 Унимодални и мултимодални приступ . . . 3
1.2 Технике препознавања емоција коришћењем неуронских мрежа . . . 3
1.3 Примене препознавања емоција . . . 4
1.4 Циљ и структура рада . . . 5

2. Основе неуронских мрежа за препознавање емоција **6**
2.1 Конволуционе неуронске мреже . . . 6
2.1.1 Математичка дефиниција конволуције . . . 6
2.1.2 Конволуциони слој у неруонским мрежама . . . 7
2.1.3 Конволуционе неуронске мреже код аудио података . . . 10
2.2 Рекурентне неуронске мреже . . . 11
2.2.1 Основна архитектура рекурентних неуронских мрежа . . . 11
2.2.2 LSTM . . . 13
2.2.3 Трансформер архитектура . . . 14
2.3 Граф неуронске мреже . . . 16
2.3.1 Опште карактеристике граф неуронских мрежа . . . 16
2.3.2 Варијације граф неуронских мрежа . . . 18
2.4 Технике оптимизације неуронских мрежа . . . 20
2.4.1 Квантизација . . . 20

3. Граф неуронске мреже за препознавање емоција **23**

4. JEPA **24**

5. Предлог рада **25**

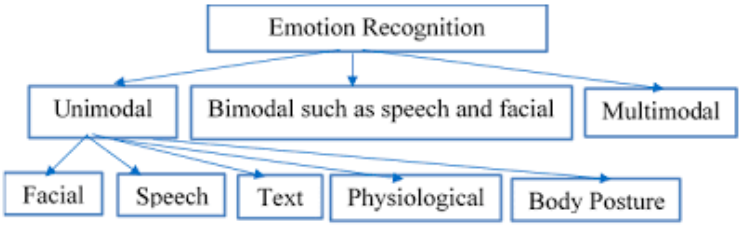
6. Литература **26**
TODO – SREDI LITERATURU . . . 26

1. Увод

Како је људски говор у великој мери мери обликован емоцијама, које утичу на тон гласа, паузе у говору, контекст разговора као и на невербалне сигнале као што је израз лица, од посебног је значаја аутоматски препознати људске емоције код система који комуницирају са људима. Управо због комплексности људских емоција и начина изражавања, аутоматско препознавање емоција је изузетно интересантна и изазовна област савремене вештачке интелигенције. У наредним поглављима увода ћемо размотрити стандардне приступе код препознавања емоција као и његове примене а затим у даљем делу детаљније обрадити моделе за препознавање емоција.

1.1 Унимодални и мултимодални приступ

Прва истраживања у овој области су се ослањала искључиво на један извор информација и то најчешће људски говор или текст, односно користила су унимодални приступ. Овај приступ је постигао одређене резултате међутим се убрзо показао ограниченим. Код текстуалног приступа проблем настаје услед чињенице да иста реченица може да носи другачије емоционално значење зависно од начина изговора. Поред тога, у току разговора, емоције једног говорника су условљене емоцијама и речима другог говорника, што унимодални приступ није био у стању да моделује. Код аудио записа говора су резултати бољи, међутим ни ту није увек могуће засигурно закључити емоције без израза лица говорника.



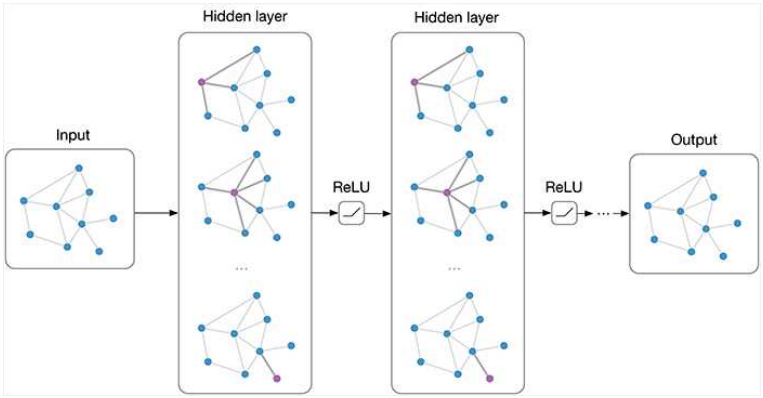
Слика 1: Подела метода препознавања емоција по броју модала

Услед ових ограничења развијени су мултимодални приступи препознавања емоција који истовремено користе више извора информација (текст, аудио и видео податке) и на тај начин боље моделују емоционално стање говорника. Модерни мултимодални системи комбинују технике обраде природног језика, рачунарског вида и обраде аудио сигнала и интегришу их у јединствену архитектуру која је способна да учи комплексне зависности из више извора података.

1.2 Технике препознавања емоција коришћењем неуронских мрежа

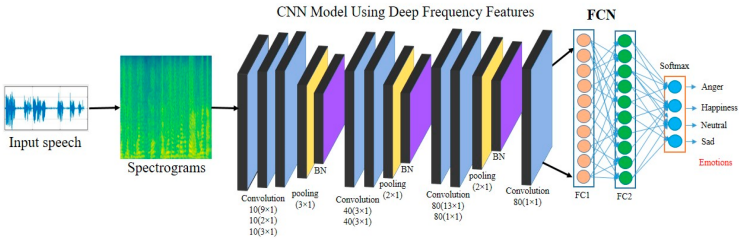
Савремене технике препознавања емоција се углавном базирају на неуронским мрежама. У овом раду ћемо обрадити две дистинктне архитектуре неуронским мрежа за препознавање емоција.

1. **Граф неуронске мреже**
   Граф неуронске мреже или ГНН су неуронске мреже специјализоване за примену на графовским подацима. Специфичност ових неуронских мрежи је *message passing*, механизам којим чворови графа међусобно размењују своје репрезентације како би се моделовале међусобне зависности чворова. У следећем поглављу ћемо детаљније обрадити како граф неуронске мреже функционишу а затим у даљим поглављима и конкретне имплементације за препознавање емоција и то прво код *COGMEN* а затим и компресоване граф неуронске мреже.



Слика 2: Граф неуронске мреже

2. **Конволуционе неуронске мреже**
   Како граф неуронске мреже могу бити доста рачунски захтевне, обрадићемо и методе које су рачунски лакше и базирају се на примени конволуционих неуронских мрежа. Поред принципа рада конволуционих неуронских мрежа описаћемо и *AVER* модел, који представља лаган конволуциони модел који препознаје емоције из аудио и видео података уз примену *federated learning*, што омогућава локалну примену овог модела, обезбеђујући брже извршење и приватност података уз цену ниже прецизности.



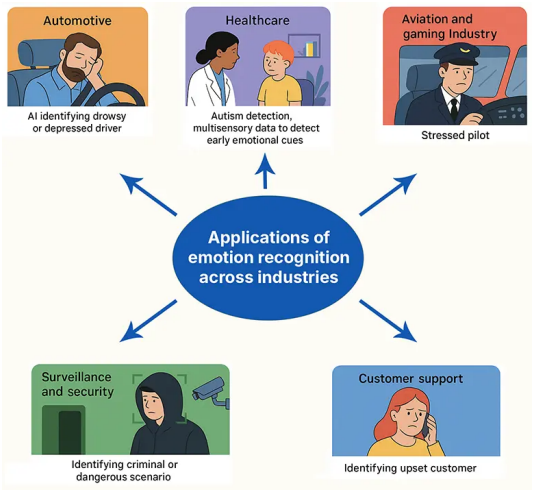
Слика 3: Конволуционе неуронске мреже за препознавање емоција

1.3 Примене препознавања емоција

Аутоматско препознавање емоција има примену у великом броју области, као што су здравство, аутомобилска индустрија, образовање, маркетингу, системима за безбедност као и многим другим.

- **Здравство**
  У здравству се системи за препознавање емоција могу користити за аутоматско праћења менталног здравља пацијената као и детекцији раних знакова различитих психичких обољења попут депресије или анксиозности
- **Аутомобилска индустрија**
  Системи за детекцију емоција се примењују на више начина у аутомобилској индустрији а један од битнијих примера је детекција умора односно поспаности или стреса код возача како би се спречиле саобраћајне несреће.

- **Образовање**
  Платформе за учење могу користити моделе за препознавање емоција како би детектовале колико су ученици укључени у учење материјала, да ли су збуњени или уморни како би кориговале технике предавања.
- **Маркетинг**
  Маркетиншке компаније могу користити аутоматско препознавање емоција како би одредиле емоције кориснике на маркетинг и на основу датих информација кориговале кампање.
- **Безбедност**
  На местима где су велике групе људи као што су тржни центри или аеродроми, могуће је користити аутоматско препознавање емоција како би се спречиле потенцијалне опасности.



Слика 4: Примена аутоматског препознавања емоција

1.4 Циљ и структура рада

Циљ овог рада је првенствено анализирати, објаснити и упоредити функционалности различитих неуронских мрежи за аутоматско препознавање емоција. Првенствено ћемо детаљно проћи кроз начин рада различитих компоненти неуронских мрежа које се користе за препознавање емоција као што су конволуциони оператор, *attention* механизам код трансформера, граф неуронске мреже и остали појмови који су неопходни за разумевање архитектуре ових модела. Затим ћемо сваки од горе поменутих модела детаљно анализирати, како његове компоненте тако и његове предности, недостатке и потенцијалне примене. На крају ћемо међусобно упоредити дата 3 модела као и обрадити могућа будућа решења и отворена питања у пољу преопознавања емоција.

2. Основе неуронских мрежа за препознавање емоција

Пре преласка на имплементацију конкретних модела за препознавање емоција ћемо објаснити појмове, технике и архитектуре модела неопходне за разумевање модела у наредном поглављу. Кренућемо од конволуционих неуронских мрежа, где ћемо видети шта је тачно конволуција у математичком смислу а затим како је примењена за екстракцију *feature*а код аудио и визуелних података. Како аудио записи говора и звука генерално нису фиксних димензија, односно модели који раде са говором, текстом или снимцима морају бити у стању да прихвате улазе различитог трајања, потребно је разумети **рекурентне неуронске мреже** што ће нас на крају довести до трансформер архитектуре и *attention* механизма. Последњу архитектуру коју ћемо обрадити, пре преласка на федерализовано учење су граф неуронске мреже које се и могу сматрати генерализацијом конволуционих и рекурентних неуронских мрежа.

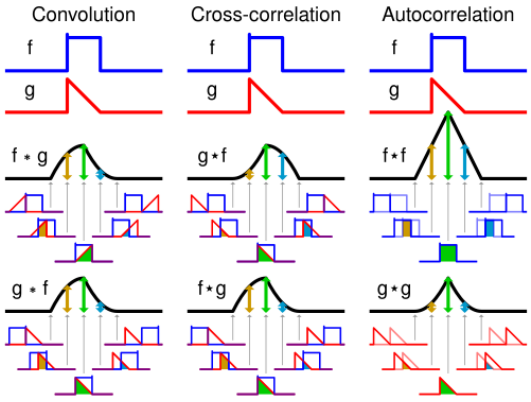
2.1 Конволуционе неуронске мреже

2.1.1 Математичка дефиниција конволуције

Математички, конволуција је операција дефинисана над две функције f и g која производи трећу функцију $f * g$ која представља интеграл производа две функције након што се једна рефлектује око y осе и помера преко друге фунцкије, односно

$$f * g = \int_{-\infty}^{\infty} f(\tau)g(t-\tau)d\tau$$

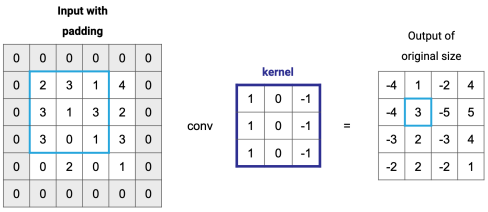
Интуитивно, конволуција се може разумети као операција којом се мери преклапање двеју функција. У процесу конволуције, функција f се фиксира, док се функција g помера слева удесно и у сваком кораку се сумира производ вредности функција.



Слика 5: Конволуција, унакрсна корелација и аутокорелација

Горе дата дефиниција се односи на непрекидне функције. Међутим, подаци у рачунару над којима ћемо примењивати моделе (аудио, видео) су дискретни. У случају дискретних функција f и g које су дефинисане над скупом комплексних бројева $\mathbb{Z}$ дефиниција конволуције гласи

$$(f * g)[n] = \sum_{m=-\infty}^{\infty} f[m]g[n-m]$$



Слика 6: Дискретна конволуција

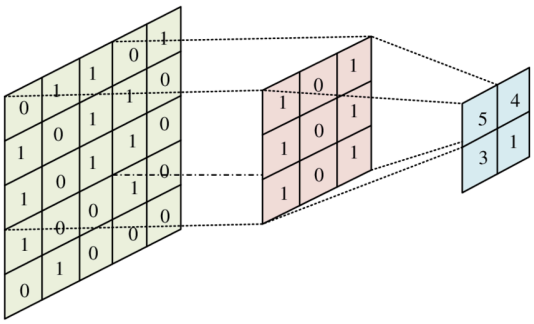
2.1.2 Конволуциони слој у неруонским мрежама

Пре настанка конволуционих мрежа, за обраду слика неуронским мрежама се користила техника поравнања (*flattening*) којом се пиксели дво-димензионалне матрице (слике) претварају у низ пиксела који се затим доводе на улазе неуронске мреже. Наравно, овом техником се губе позиционална зависност између пиксела, па су овакви модели имали знатно горе перформансе.

Као решење овог проблема су током 90-их година прошлог века уведене конволуционе неуронске мреже, базиране на математичкој конволуцији обрађеној у претходној области. Развојем наредних деценија конволуционе мреже су напредовале али главни принцип рада је одржан. Разлог популарности конволуционих мрежа је поред добрих резултата су и перформансе. Како се применом коволуционог слоја издвајају *feature*-и на слици тиме се и смањује број параметра модела, омогућавајући примену неуронским мрежа и на великим сликама.

Конволуциони слој и кернел

Конволуциони слојеви у неуронским мрежама су слојеви који примењују конволуцију улазног сигнала и више задатих кернела. Кернел, такође познат као филтер је матрица бројева која се помера дуж улазног сигнала и користи за рачунање конволуције. Након конволуције се излазној вредности додаје и *bias*, а затим над излазом конволуционог слоја се примењује активациона функција (нпр *ReLu*).



Слика 7: Примена конволуције над сликама

Ручно одредити сваки кернел је немогућ задатак, тако да се заправо тежине свих филтера у конволуционим слојевима уче у току тренирања неуронске мрежа. На крају је сваки кернел задужен за детекцију одређеног *feature*а у улазном податку.

Излаз неурона i, j у слоју l је дат формулом

$$z_{i,j}^{(l)} = \sum_{m,p,q} W_{p,q,m}^{(l)} a_{is+p,\, js+q,\, m}^{(l-1)} + b^{(l)}, \qquad a_{i,j}^{(l)} = f\left(z_{i,j}^{(l)}\right)$$

где W представља филтер, s корак (stride), а f активациону функцију.

Током тренирања потребно је одредити градијенте функције губитка $\mathcal{L}$ по свим параметрима мреже. Интуитивно, градијент по параметру $W_{p,q}$ одређује колико свака од вредности кернела утиче на коначну грешку модела и може се представити на следећи начин

$$\frac{\partial \mathcal{L}}{\partial W_{p,q,m}^{(l)}} = \sum_{i,j} \delta_{i,j}^{(l)} \cdot a_{is+p,\, js+q,\, m}^{(l-1)}$$

где је

$$\delta_{i,j}^{(l)} = \frac{\partial \mathcal{L}}{\partial z_{i,j}^{(l)}}$$

грешка на позицији i, j у слоју l, добијена из наредног слоја. Градијент по *bias*у своди се на просто сабирање свих грешака:

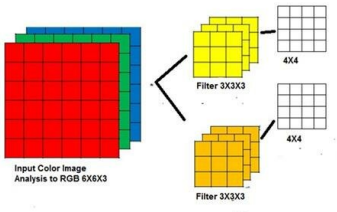
$$\frac{\partial \mathcal{L}}{\partial b^{(l)}} = \sum_{i,j} \delta_{i,j}^{(l)}$$

Када су градијенти познати, параметри се ажурирају у смеру супротном од градијента:

$$W_{p,q,m}^{(l)} \leftarrow W_{p,q,m}^{(l)} - \eta \cdot \frac{\partial \mathcal{L}}{\partial W_{p,q,m}^{(l)}}, \qquad b^{(l)} \leftarrow b^{(l)} - \eta \cdot \frac{\partial \mathcal{L}}{\partial b^{(l)}}$$

где је η стопа учења (*learning rate*).

Конволуцију можемо применити над подацима различитих димензија, при чему улаз и кернел морају бити исте димензионалности. У случају временских података (нпр аудио сигнал у временском домену) примењујемо 1D конволуцију, код монохроматских слика 2D док код слика са више канала (слике у боји) или снимака користимо конволуцију са више димензија. У 3D случају, за сваки филтер, сваки канал једног филтера се примењује над одговарајућем каналу улаза и вредности се сумирају.



Слика 8: RGB конволуција

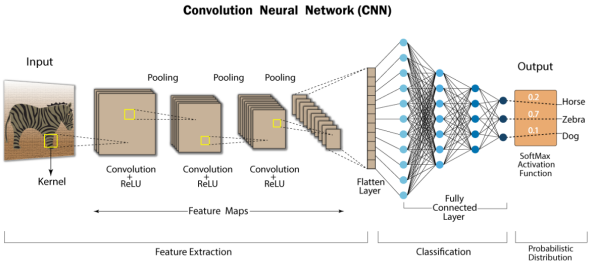
За дефинисање конволуционог слоја се узима више улазних параметара

- **Величина кернела**
- ***Stride***
  Представља број јединица за које се врши транслација филтера у односу на улазни податак током конволуције.
- ***Padding***
  Представља број додатних пиксела којим ће се оивичити улазни податак. Циљ је одржати једнаким величине улаза и излаза, као и дати подједнаку важност ивичним подацима.

Комбинацијом вредности ових параметара се одређује и величина излаза конволуционог слоја, који се затим доводи као улаз *pooling* слоју.

***Pooling* слој**

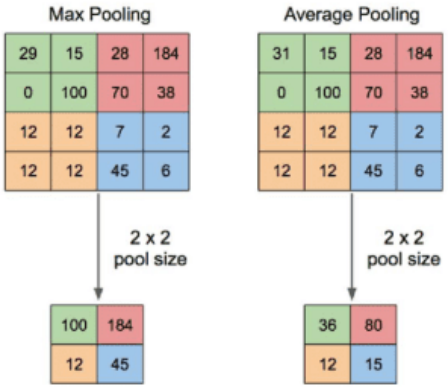
Сваким конволуционим слојем кроз који сигнал пролази величина се смањује и постепено добијају *feature*и улазног сигнала који се затим користе као улази вишеслојног перцептрона за коначно предвиђање. Битна особина конволуционих слојева јесте позициона инваријатност, тј где год се *feature* појавио у сигналу детектоваће се применом конволуције над целим улазом. Ово је потпомогнуто смањивањем сигнала које смо поменули, с обзиром да тиме мрежа учи генералније *feature*е и не фокусира се на ситне детаље.



Слика 9: Архитектура конволуционе мреже

Како би даље ојачали ту особину као и даље смањили излаз конволуционих слојева уводи се ***pooling* слој**. *Pooling* је операција којом се узима свака подматрица величине nxn а затим изводи математичка операција над вредностима чији резултат затим представља вредност излазне матрице. Углавном је то max или $average$ операција.

Можемо да приметимо сличност са конволуцијом где примењујемо max или $average$ у оквиру неког филтера. Зато као и код конволуције можемо специфирати величину филтера, *padding* и *stride*.



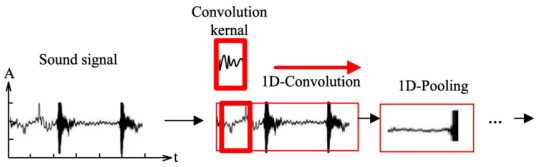
Слика 10: Pooling операција

2.1.3 Конволуционе неуронске мреже код аудио података

Конволуционе неуронске мреже су првенствено осмишљене за рад са сликама (за препознавање цифара) а касније су примењене и код аудио података. Код аудио података могуће је вршити 1D конволуцију над аудиом у временском домену или 2D конволуцију код звука у фреквентном домену.

1D Конволуција

Када је аудио дат у временском домену, где је звук описан амплитудом која се мења у времену, користимо једнодимензионалну конволуцију. Применом конволуције се препознају временски *feature*и и мреже са оваквим типом конволуције су погодне за задатке који не захтевају препознавање фреквентних зависности, као што су сепарација извора звука или компресија звука, наручито на уређајима са слабијим процесорима.

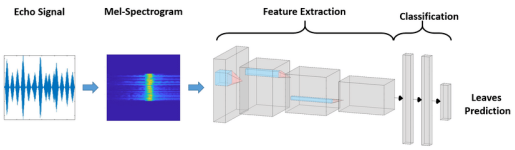


Слика 11: 1D конволуција звука

Пример оваквих модела је *WaveNet* који служи за генерисање људског говора.

2D Конволуција

Када нам је важно како се фреквентни садржај звука мења током времена, можемо применити 2D конволуцију. Прво је потребно претворити звук у фреквентни домен, односно одредити *mel* спектрограм звука. Тада можемо применити конволуцију слично као код слика. Овакве неуронске мреже се могу користити за детекцију специфичних догађаја у звуку (детекција кључних речи, позадинских звукова итд.).

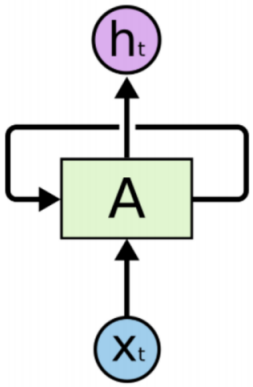


Слика 12: 2D конволуција звука

Пример оваквих модела је *VGGish* који служи за издвајање аудио *feature*а на основу *mel* спектрограма звука.

2.2 Рекурентне неуронске мреже

Код стандардних неуронских мрежа, димензије података које мрежа обрађује морају бити познате унапред. Код података који могу бити различитих димензија (нпр. текст различитих дужина, звук различитог трајања) ово представља проблем. Као решење овог проблема су се јавиле рекурентне неуронске мреже, које имају повратне гране у скривеним слојевима. На овај начин, мрежа одржава скривено стање које сумаризује претходно виђене улазе, чиме се омогућава обрада секвенци произвољне дужине корак по корак.



Слика 13: Ћелија рекурентне неуронске мреже

У наредним поглављима ћемо анализирати основне рекурентне неуронске мреже као и проблеме који настају њиховим тренирањем а затим и *LSTM* архитектуру која је решила неке од тих проблема и дуго било стандардна мрежа за обраду секвенцијалних података. На крају ћемо обрадити и трансформере који су довели до *LLM* револуције у току последње деценије.

2.2.1 Основна архитектура рекурентних неуронских мрежа

Најосновнија рекурентна неуронска мрежа, као и друге неуронске мреже, на улазу добија сигнал x и генерише излаз y. Разлика у односу на стандардне неуронске мреже јесте да се резултат y добија не само на основу улаза x и тежина мреже већ и на основу претходног излаза мреже. Као интуитивно полазиште, излаз мреже у тренутку t може се посматрати као функција тренутног улаза и претходног

излаза, односно

$$y_t = \psi(W_x^T x_t + W_y^T y_{t-1} + b)$$

при чему је ψ активациона функција.

Међутим, у пракси се уводи посебна функција скривеног стања h_t која моделује интерно стање мреже одвојено од излаза, чиме се омогућава комплексније моделовање на основу улаза и претходног стања

$$h_t = f(x_t, h_{t-1})$$

На овај начин се предвиђање излаза мреже и памћење претходног стања може раздвојити

$$y_t = \psi(W_h y^T h_y + b)$$

$$h_t = \psi(W_x^T x_t + W_x h^T h_{t-1})$$

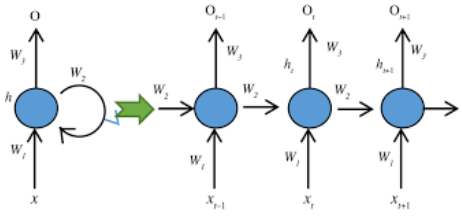
Уколико покушамо да тренирамо овакву рекурентну неуронску мрежу било би потребно обавити n пролаза кроз мрежу како би добили резултате за целу секвенцу а затим вршити *backpropagation* кроз време, тј променити тежине тако да се у скривеном стању мреже енкодирају потребне информације за инференцију.

У случају $h_t = w \cdot h_{t-1}$ применом ланчаног правила добијамо

$$\frac{\partial h_t}{\partial w} = h_{t-1} + w \cdot \frac{\partial h_{t-1}}{\partial w} = h_{t-1} + w \cdot \left(h_{t-2} + w \cdot \frac{\partial h_{t-2}}{\partial w} \right) = ...$$

односно када постоји петља можемо заувек кружити, односно нема базног случаја.

Решење овога проблема је претворити граф који описује рекурентну неуронску мрежу у ациклични граф, процедуром која се назива *unrolling*. Овај процес избацује циклус тако што прави *n* копија мреже, тако да i-та копија прима улаз x_i а на излазу даје y_i и h_i који се затим води на улаз $i+1$ копије. Такође, овим процесом дефинишемо базни случај што омогућава *backpropagation*.



Слика 14: Unrolling

Проблем који овде настаје јесте када желимо мрежу да користимо за дуге секвенце. Размотримо случај када имамо само 100 копија, што је у реалним случајевима и кратко. Грешку коју добијемо на крају мреже пропагирамо кроз мрежу до првог слоја и применом ланчаног правила добијамо израз

$$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial h_{100}} \cdot \frac{\partial h_{100}}{\partial h_{99}} \cdot \frac{\partial h_{99}}{\partial h_{98}} \cdot ... \cdot \frac{\partial h_2}{\partial h_1}$$

У рекурентним неуронским мрежама се користи активациона функција при рачунању скривеног стања. Погледаћемо *tanh* као пример. Извод ове функције има максималну вредност 1 и то само у случају када је улаз једнак 0. Уколико је први извод сваког од скривених стања био чак и релативно висок, услед великог броја множења може доћи до **нестајања градијента** јер $0.9^{100} \approx 0.0000265$. Тренирање нижих слојева је тиме практично немогуће.

Сличан проблем настаје и код активационих функција где извод може бити већи од 1. Ако је извод и мало већи, рецимо 1.1, добићемо да је извод за прву копију $1.1^{100} \approx 13780$. Овај проблем се назива **експлозија градијента**.

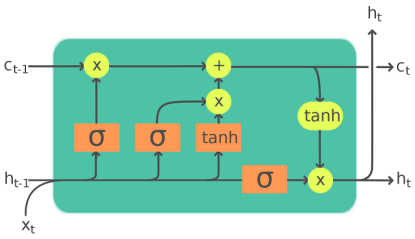
Због ова два проблема класичне рекурентне мреже нису нашле примену ван врло кратких секвенци. Као бољи модел, отпоран на експлозију и нестанак градијената, се јавио *LSTM*.

2.2.2 LSTM

Пре преласка на трансформере размотрићемо *LSTM* архитектуру као међукорак. Као што смо видели у претходном делу, класичне рекурентне мреже имају проблема приликом тренирања над дужим секвенцама. Као решење овог проблема су се јавиле *LSTM* мреже које раде на следећи начин.

Као и класичне рекурентне мреже и *LSTM* има x_t на улазу и на излазу h_t који служи за генерисање излаза. Поред тога ћелије у *LSTM* мрежи поседују још једно скривено стање које служи као интерна меморија мреже.

У тренутку t ћелија ће на улазу имати x_t, h_{t-1} и C_{t-1}, при чему h_{t-1} је излаз претходне ћелије док је C_{t-1} интерно стање мреже у претходном тренутку.



Слика 15: LSTM ћелија

LSTM ћелија има 3 главна *gate*а

1. ***Forget gate***
   У првом кораку се одређује колико ће се претходно стање C_{t-1} обрисати, односно заборавити

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

2. ***Input gate***
   У следећем кораку се одређује како треба ажурирати скривено стање C_t. Рачунају се две вредности

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$$

А затим се рачуна ново стање

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

3. ***Output gate***
   На крају се рачуна h_t на следећи начин

$$h_t = o_t * tanh(W_O[h_{t-1}, x_t] + b_O)$$

Оваква имплементација омогућава да се чува само релевантан контекст и тиме се знатно ублажава проблем нестајућих и експлодирајућих градијената. Због овога су *LSTM*ови у стању да раде са много дужим секвенцама.

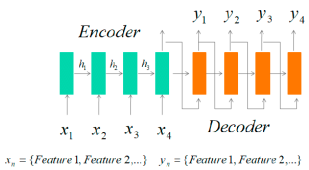
Ограничење ове мреже је што скривено стање *n*-те ћелија у мрежи је највише одређено на основу претходне ћелије, односно претходног елемента секвенце, а најмање првим елементом секвенце, односно губи се контекст са повећањем броја ћелија. Због тога *LSTM*ови нису у стању да детектују зависности између удаљенијих делова секвенце, што је круцијално код обраде текста и детекције емоција. Овај проблем је решен у трансформер архитектури.

2.2.3 Трансформер архитектура

Seq2Seq модели

На почетку 2010-их доминантни модели за обраду природних језика су били сачињени од *LSTM* ћелија које су градиле енкодерске и декодерске мреже.

- *Encoder*
  Прву мрежу представља енкодер, имплементиран као низ *LSTM* ћелија, који на улазу добија елементе секвенце и производи вектор који треба да енкапсулира контекст унете секвенце. Овако добијен вектор се може користити у вишеслојном перцептрону за задатке као што је анализа сентимента или довођењем на декодер за генеративне задатке.
- *Decoder*
  Вектор произведен од стране енкодера се доводи на улаз декодера који затим производи излаз. Енкодер декодер мреже су посебно биле корисне за превођење језика и генерисање текста, међутим највећи проблем су и даље представљале дуже секвенце услед ограничења *LSTM*ова као и чињеница да је немогуће задржати целокупан контекст дужег текста у оквиру једног вектора.



Слика 16: $Seq2Seq$

Attention механизам

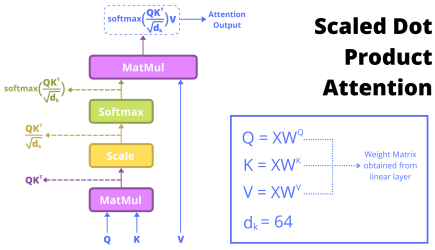
Као решење овог проблема се јавио *attention* механизам. Циљ овог механизма је да омогући моделу да учи зависност сваког излаза мреже од сваког претходног улаза, на тај начин решавајући проблем губљења контекста код дужих секвенци. Интуитивно, овај механизам се може схватити као процес сличан људском разумевању текста, на пример током превођења. Уколико у току читања текста наиђемо на заменицу покушаћемо да је повежемо са особом која је субјекат радње у претходном тексту, уместо са свим речима пре прочитане.

Како имплементирати овакав "фокус" математички? Трансформери имплементирају ту функционалност помоћу *query*, *key* и *value* матрица.

- ***Key (K)***
  Матрица K представља улазне елементе секвенце који су енкодирани *embedding* векторима и који садрже контекст сваког елемента секвенце. Вектори колоне матрице K се добијају као излаз енкодера и ова матрица се рачуна као
  $$K = xW_K$$
  Матрица W_K се учи у току тренирања модела.
- ***Query (Q)***
  Матрица Q представља "питања" која сваки елемент секвенце поставља а затим производом QK^T добија се мера релевантности сваког елемента секвенце у односу на сваки други елемент секвенце. Ова "питања" представљају меру различитих зависности између елемената секвенце и у области обраде природних језика примери оваквих питања могу бити провера ком глаголу припада субјекат, на кога се односи заменица итд.
  $$Q = xW_Q$$
  Матрица W_Q се учи у току тренирања модела.
- ***Value (V)***
  Матрица V садржи излазне *embedding* векторе и дата једначином
  $$V = xW_V$$
  при чему се током тренирања матрица W_V учи.

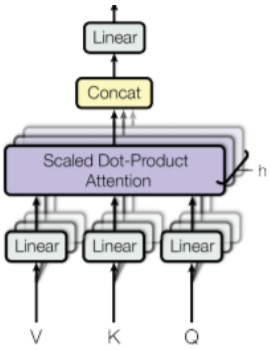
На основу претходне три матрице добијамо *attention score*, односно на које делове секвенце треба обратити пажњу, тако што се множи Q и K^T, резултат се скалира и примењује се *softmax* како би нормализовали вредности у вероватноће. Дељење са $\sqrt{d_k}$ спречава засићење *softmax* функције које се јавља када су вредности производа QK^T превелике, што би довело до нестајања градијената.

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



Слика 17: *Attention* механизам

Као унапређење *attention* механизма, јавио се *multi-head attention*. Коришћењем само једног *attention* слоја омогућава се моделу да учиједну врсту зависности између делова секвенци, тако да додавањем више оваквих слојева који примају исти улаз али имају своје тежине омогућава се учење више различитих зависности.



Слика 18: *Multihead attention*

У зависности од тога да ли користимо *attention* над деловима исте секвенце или над деловима неке друге секвенце, овај механизам се дели на *self-attention* и *cross-attention*, респективно.

Како трансформери немају рекурентност, редослед елемената секвенце се уноси кроз позиционо кодирање које се додаје на *embedding* векторе пре уласка у мрежу.

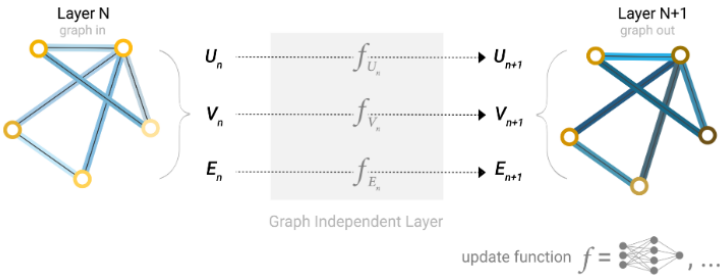
2.3 Граф неуронске мреже

У наредним поглављима размотрићемо граф неуронске мреже које се могу користити код задатака са графовским подацима, као што су анализа друштвених мрежа, предвиђање својстава молекула и генерално задаци где се између података могу установити релације које се могу затим представити графом.

2.3.1 Опште карактеристике граф неуронских мрежа

Најосновније граф неуронске мреже на улазу примају граф и на излазу генеришу трансформисане репрезентације чворова, ивица и глобалних карактеристика графа, добијене применом сукцесивних трансформација без промене структуре улазног графа. Трансформације се врше над 3 компоненте графа.

- Ивице (E - *Edges*)
  Матрица ивица чува особине ивица, које могу бити задате скаларом али најчешће се представљају *embedding* векторима. Пример оваквог *embedding* вектора је код предикције својстава молекула, при чему матрица ивица енкодира информације о хемијским везама (тип везе, растојање између атома, итд.) за сваку од ивица.
- Чворови (V - *Vertices*)
  Матрица чворова V чува особине чворова, које могу такође бити задате скаларом али најчешће се представљају *embedding* векторима, као и код ивица. Један пример је код анализе друштвених мрежа, где матрица чворова енкодира информације о корисницима (пол, старост, интересовања итд.) за сваки чвор.
- Универзалне карактеристике графа (U - *Universal*) Енкодира универзалне карактеристике графа које се могу на крају користити за предикцију својстава графа. Овде се енкодирају карактеристике као што су глобална структура графа, повезаност итд.



Слика 19: Граф неуронска мрежа

Најпростија граф неуронска мрежа дефинише по један вишеслојни перцептрон за сваки од улаза U, V и E и на излазу генерише ажуриране верзије ових матрица. На основу коначних матрица се може вршити предикција на нивоу чворова, ивица или на нивоу графа. Проблем који се јавља код овакве архитектуре је код задатака где је потребно анализирати једну од 3 компоненте графа (нпр. чворове), а имамо податке о другим компонентама (нпр. ивице). Решење овог проблема је *pooling*, који се врши у два корака

1. **Прикупљање** *embeddinga* информација које желимо да проследимо и конкатенација у матрицу
2. **Агрегација** скупљених *embeddinga*, најчешће сумирањем.

Код примера анализе чворова без информација о њима, могуће је прикупити репрезентације ивица и затим у чворовима сумирати репрезентације суседних ивица сваког чвора. У даљим примерима ће ова операција бити означена са

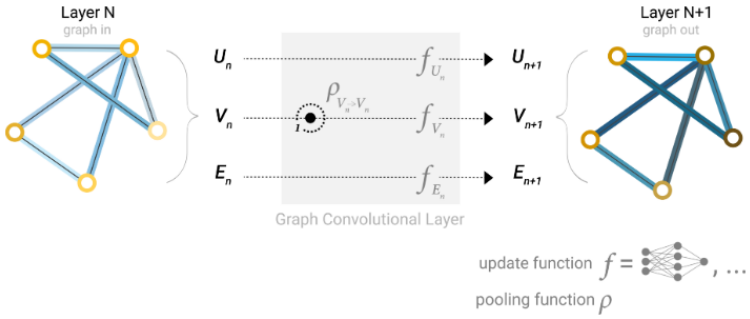
$$\rho_{X \to Y}$$

што означава *pooling* података X (нпр. чворови) у Y (нпр. ивице).

Овакав *pooling* је могуће применити у сваком слоју граф неуронске мреже и то и над истим подацима. Оваквом техником се омогућава дељење информације између суседних чворова или суседних ивица у графу, енкодирајући у *embedding* векторима и конективност графа. Овакав механизам, са додатим учењем ажурирања репрезентација, се назива *message passing* и обавља се у три корака

1. За сваки чвор или ивицу скупити све репрезентације суседних чворова или ивица.
2. Агрегирати све репрезентације функцијом агрегације (као сумирање).
3. Прикупљене репрезентације пролазе кроз *update* функцију, најчешће неуронску мрежу.

Понављањем *message passing*а n пута могуће је довести до сваког чвора податке из чворова удаљених n корака.



Слика 20: Message passing

2.3.2 Варијације граф неуронских мрежа

Граф конволуционе и релационе конволуционе неуронске мреже

Како се конволуција показала изузетно успешном код задатака из рачунарског вида и обраде звука, следило је да би потенцијално била применљива и код графовских података. Примена конволуције над графовима је нешто другачија и дефинише се преко Лапласијана графа

$$L = D - A$$

Ова операција је изузетно скупа, па је креирана апроксимација преко нормализоване агрегације суседа

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}H^{(l)}W^{(l)}\right)$$

при чему је

- $H^{(l)}$
  матрица репрезентација чворова у слоју l. Сваки ред ове метрице је *embedding* вектор једног чвора.
- $W^{(l)}$
  матрица тежина слоја l. Ове тежине се уче током тренирања.
- $\tilde{A} = A + I$
  представља матрицу суседства у графу са додатим повратним везама, тј за сваки чвор у графу се додаје ивица која повезује чвор са самим собом.
- $\tilde{D}$
  је дијагонална матрица где је $\tilde{D}_{ii}$ степен чвора i, тј број суседа. Ова матрица служи за нормализацију агрегације јер чворови са великим бројем суседа би имали велике вредности након агрегације.

- σ
  нелинеарна активациона функција, нпр *ReLU*.

Како можемо видети, за све ивице у слоју l се користи иста матрица тежина $W^{(l)}$. Ово значи да су стандардне граф конволуционе неуронске мреже у стању да уче са графицима који имају само једну врсту релација између чворова. Ово решавају релационе граф конволуционе мреже адаптацијом горњег израза, следећим изразом

$$h_i^{(l+1)} = \sigma\left(W_0^{(l)} h_i^{(l)} + \sum_{r \in R} \sum_{j \in N_i^r} \frac{1}{c_{i,r}} W_r^{(l)} h_j^{(l)}\right)$$

- $h_i^{(l+1)}$
  репрезентација чвора i у слоју $l+1$
- $W_0^{(l)}$
  матрица тежина за сопствене конекције, услед чега се у изразу множи за репрезентацијом чвора i у слоју l
- $r \in R$
  r представља један тип релације између чворова, док је R скуп свих могућих релација
- $j \in N_i^r$
  j представља чвор који дели ивицу са чвором i, при чему ивица представља релацију r
- $W_r^{(l)}$
  матрица тежина за релацију r
- $c_{i,r}$
  нормализациона константа, често подешена на $|N_i^r|$ односно број суседа са којима тренутни чвор i има релацију r

Граф трансформери

Код стандардних граф неуронских мрежа користимо *message passing* механизам који смо обрадили у претходном поглављу. Можемо повући паралелу између овог механизма и начина рада стандардних рекурентних неуронских мрежа.

- Код рекурентних неуронских мрежа, да би у n-том елементу секвенце добили информацију о првом елементу, прво морамо проћи кроз првих $n-1$ корака где ћемо у сваком енкодирати у скривеном стању претходно обрађене податке.
- Код граф неуронских мрежа, да би у неком чвору или ивици графа добили информацију о чвору или ивици удаљеним n корака, прво морамо проћи кроз чворове удаљене $1, 2, ... n-1$ корака.

Као и код рекурентних мрежа овде се јављају пар проблема

- **Деградација контекста**
  Уколико желимо да мрежа научи зависности између удаљених делова графа (секвенце код *RNN*) морамо проследити репрезентације у више корака, чиме се губи контекст, односно подаци о удаљеним чворовима или ивицама. Сличан проблем се јавља и када чвор има пуно суседа, при чему се агрегацијом нужно губе подаци о суседима (тешко је агрегирати све репрезентације великог броја суседа у један вектор.)

- ***Oversmoothing***
  Приликом размене репрезентација између чворова врши се њихова агрегација (сумирање) и при-

мена активационе функције. Понављањем размена порука, репрезентације чворова постају сличније тј конвергирају ка истој вредности. Тиме се опет губи контекст током тренинга.

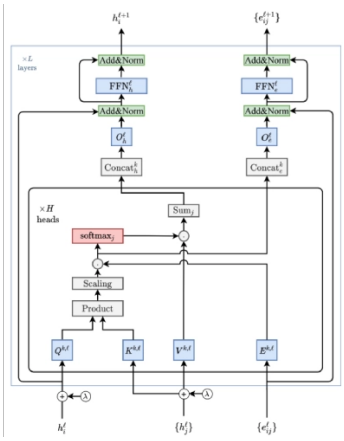
Слично решење се може применити као и код рекурентних мрежа, тј. увести *attention* механизам. У оваквим граф неуронским мрежама се као матрица упита Q користи *embedding* вектор чвора i u тренутку l

$$h_i^l$$

док се као матрице K и V користе *embedding* вектори суседних чворова у тренутку l

$$h_j^l$$

Уколико имамо податке и о ивицама између ових чворова могу се искористити пре примене $softmax$ операције. На излазу се добијају ажуриране репрезентације чворова и ивица. На овај начин се могу анализирати појединачне зависности између удаљених делова графова, док додавањем више *attention head*а ова архитектура је у стању да учи више различитих зависности.



Слика 21: Граф трансформер неуронска мрежа

2.4 Технике оптимизације неуронских мрежа

Тренинг и инференција неуронских мрежа су скупе операције, због чега се велики модели углавном налазе на серверима са моћним хардвером. Међутим, код неких проблема потребно је покренути моделе на уређају као што су *embedded* или мобилни уређаји који имају знатно мање меморије и слабим процесорима. У наредном поглављу ћемо размотрити квантизацију, технику оптимизације инференције неуронских мрежа.

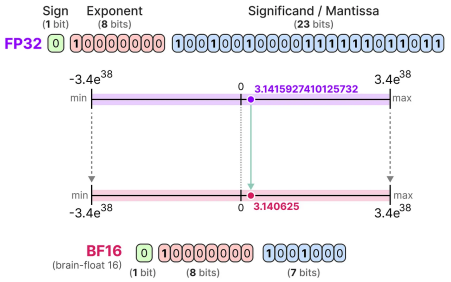
2.4.1 Квантизација

Комплексније неуронске мреже могу имати велики број слојева а тиме и велики број тежина које се морају учитати у меморију током инференције модела. Ако сваку тежину представимо као $float32$ ili $float64$ вредности како би очували прецизност тежина, свака тежина ће заузимати 4 или 8 бајта. Са великим бројем тежина долазимо до ситуације да је практично немогуће сместити модел у меморију на слабијем хардверу.

Једно решење овог проблема је смањити прецизност података у којим се складиште тежине. Овим се смањује меморијска и процесорска захтевност модела али штети прецизности, с тим да смањење у прецизности је често прихватљиво у хардверски ограниченим уређајима.

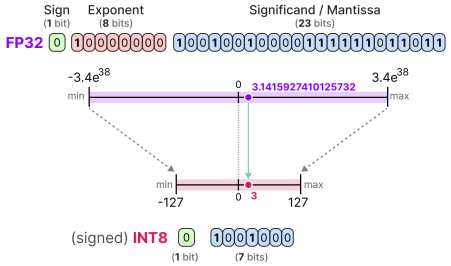
Квантизацију најчешће вршимо конверзијом $float32$ или $float64$ тежина у

- ***bfloat16*** С обзиром да конверзијом са прецизнијих *float* формата у $fp16$ долази до значајног губитака информација осмишљен је формат *bfloat16* којим је промењена структура *float16*
  - Задржава се један *sign* бит
  - **Експонент** уместо 5 битова добија сада 8 битова као код *fp32* како би имали исти распон вредности
  - **Мантиса** уместо 10 битова сада има 7 битова На овај начин се конверзијом смањује прецизност али задржава распон вредности.



Слика 22: $bfloat16$

- **int8** Даљим смањивањем броја битова за чување тежина долазимо до квантизације на 8 битова. Ова квантизација доводи до смањења величине тежина чак 4 пута као и убрзања рачунања, с обзиром да неке врсте хардвера имају брже *integer* операције. Наравно следује и највећи пад у прецизности модела овом квантизацијом



Слика 23: $int8$

Код *bfloat16* квантизација се врши директним одсецањем вредности док код случаја са *int8* квантизацијом примењује се мапирање *float32* вредности на *int8*, након чега је потребно и деквантизовати вредности. Постоје два стандардна начина мапирања

- **Симетрична квантизација**
  Симетричном квантизацијом се све вредности у распону *fp32* мапирају на *int8* вредности симетрично, што значи да најмања и највећа могућа вредност које се квантизују су једнаке по апсолутној вредности и вредност 0 остаје 0 након квантизације.

За скалирање се рачуна фактор скалирања

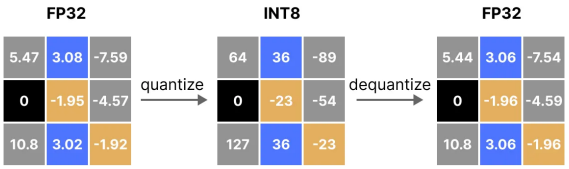
$$s = \frac{2^{b-1} - 1}{\alpha}$$

при чему је b број битова након квантизације а α највећа апсолутна вредност од датих *fp32* вредности. Како би добили квантизовану вредност користимо формулу

$$x_q = round(s \cdot x)$$

а касније за деквантизацију

$$x_d = \frac{x_q}{s}$$



Слика 24: Квантизација и деквантизација

- **Асиметрична квантизација**
  Претходни тип квантизације је једноставнији, али има недостатак у томе што ако имамо вредности од 0 до 1 као тежине, квантизацијом ћемо мапирати распон $[-1, 1]$, беспотребно губећи више прецизности него потребно. Асиметрична квантизација зато бира најмању вредност међу подацима β и највећу α и затим мапира вредности у опсегу $[-\beta, \alpha]$ на опсег $[-128, 127]$
  Код асиметричне квантизације потребно је одредити фактор скалирања као и вредност у коју се мапира 0 вредност у оригиналном распону.

$$s = \frac{255}{\alpha - \beta}$$

$$z = round(-s \cdot \beta) - 2^{b-1}$$

Тада је квантизована вредност

$$x_q = round(s \cdot x + z)$$

а деквантизована

$$x_d = \frac{x_q - z}{s}$$

Проблем који настаје код обе технике квантизације јесте губитак вредности када имамо *outlier*е. На пример, ако су све вредности вектора блиске 0, док је једна доста већа све вредности ће бити анулиране док ће *outlier* имати максималну вредност распона. Због овога се врши *clipping* којим одсецамо вредности оригиналног вектора. Ако се ради *clipping* у распону $[-\alpha, \alpha]$ онда ће све вредности мање од $-\alpha$ бити замењене са $-\alpha$ док ће веће од α бити замењене са α.

Квантизацију је могуће вршити током тренинга или након тренинга, при чему квантизација у току тренинга даје боље резултате с обзиром да је модел у стању да узме у обзир ефекат квантизације.

3. Граф неуронске мреже за препознавање емоција

Претходни модел који смо анализирали је постизао одличне резултате код *embedded* уређаја, међутим могуће је развити комплексније моделе са бољим перформансама. У следећем поглављу размотрићемо модел дат у раду *COGMEN:COntextualized GNN based Multimodal Emotion recognitioN*, који је базиран на граф неуронским мрежама и трансформер архитектури. Овај модел постиже тренутно најбоље резултате у мулдимодалном препознавању емоција.

[ERROR: Cannot read file: sections/3_emotion_gnn/1_motivation/main.md]

[ERROR: Cannot read file: sections/3_emotion_gnn/2_architecture/main.md]

[ERROR: Cannot read file: sections/3_emotion_gnn/3_results/main.md]

[ERROR: Cannot read file: sections/3_emotion_gnn/4_optimization/main.md]

4. JEPA

5. Предлог рада

6. Литература

TODO – SREDI LITERATURU

1. https://milvus.io/ai-quick-referencehow-can-convolutional-neural-networks-cnns-be-applied-to-audio--data
2. https://medium.com/@RobuRishabh/recurrent-neural-network-rnn-8412b9abd755
3. https://medium.com/@fredroj/recurrent-neural-network-rnn-cell-internal-mechanism-ff5eca732adb
4. https://www.geeksforgeeks.org/machine-learning/introduction-to-recurrent-neural-network/
5. https://colah.github.io/posts/2015-08-Understanding-LSTMs/
6. https://poloclub.github.io/transformer-explainer/
7. https://medium.com/data-science/introduction-to-attention-mechanism-in-deep-learning-eli5-way--d6c2d799ef18
8. https://medium.com/@paulpierre/eli5-transformers-gpt-and-the-power-of-the-attention-mechanic--6504693f2c3f
9. https://www.d2l.ai/chapter_attention-mechanisms-and-transformers/queries-keys-values.html
10. https://apxml.com/courses/introduction-to-transformer-models/chapter-2-self-attention-multi-head-attention/query-key-value-vectors
11. https://kumo.ai/research/introduction-to-graph-transformers/
12. https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-quantization
13. https://www.dgl.ai/dgl_docs/tutorials/models/1_gnn/4_rgcn.html